Week 3

Search I: Systematic Search

◎ Learning Objectives

By the end of this chapter, you will be able to:

- **(H1)** explain what backtracking search is, construct or trace a search tree by hand for a small CSP (root, choice points, branches, dead ends, solution leaf), and explain why systematic backtracking is *complete* – guaranteed to find a solution if one exists or correctly report infeasibility – even though it only checks constraints between already-assigned variables;
- **(H2)** compare variable-ordering heuristics (MRV, the degree heuristic, dom/wdeg) and the value-ordering heuristic LCV, explain the "fail-first" versus "succeed-first" intuition each embodies, and select an appropriate variable/value ordering for a given small CSP by hand;
- **(H3)** explain forward checking and MAC (maintaining arc consistency during search) as two look-ahead techniques of different strength, and trace, by hand, their effect on neighbouring variables' domains at a single search-tree node;
- **(H4)** configure a CP-SAT model's decision strategy (`AddDecisionStrategy`), worker count (`num_search_workers`) and time limit (`max_time_in_seconds`), run the model, and read the solver's search statistics (`NumBranches`, `NumConflicts`, `WallTime`, `StatusName`) to compare two configurations of the same model;
- **(H5)** given a hard instance from this week's problem pool (graph colouring, magic square or Langford pairs), choose and implement, in CP-SAT, a variable/value-ordering-plus-look-ahead configuration suited to it, and use the solver's own search statistics to measure and explain the resulting difference in solving effort compared with CP-SAT's default configuration.

Week 1 built paper and CP-SAT models and treated the solver as a black box that simply returned a solution. Week 2 opened that box halfway, showing how propagation narrows every domain *before* any choice is made. This week opens it the rest of the way: the actual choose-a-variable, choose-a-value, backtrack-on-failure loop that runs underneath every CP-SAT call, the classic heuristics that decide what that loop tries first (MRV, the degree heuristic, dom/wdeg, LCV), the classic look-ahead techniques that decide how much of the tree the loop prunes before it even

gets there (forward checking, MAC), and the concrete CP-SAT parameters – decision strategies, worker count, time limit – that let you observe, and where useful steer, that loop yourself.

Four units carry this chapter, in a fixed order, and Units 1–3 stay deliberately on paper – hand-worked examples and figures, exactly the pattern Week 1 used for its own opening units – with Unit 4 as this week's sole code unit. Unit 1 defines backtracking search and the search tree it traces out, worked by hand on 4-queens. Unit 2 asks which variable and which value the search should try first, comparing MRV, the degree heuristic and dom/wdeg against LCV. Unit 3 asks how much work can be done *before* a doomed branch is even tried, comparing forward checking's one-hop look-ahead against MAC's full re-propagation. Unit 4 puts all three units' vocabulary onto CP-SAT's own API, using a larger 10-node graph-coloring instance, the 4×4 magic square (its first code this course, after Week 1 formulated it on paper only), and a new problem, Langford pairs, chosen specifically because its search-tree size is dramatically sensitive to variable and value order.

This chapter stays deliberately out of several topics later weeks own outright. It introduces no local search, restarts, nogood learning, large neighbourhood search or parallel-portfolio *depth* – that is Week 4's entire subject; Unit 4's light look at `num_search_workers` is only a "here is the knob and what it visibly does" preview, not an explanation of *why* parallel search helps. It introduces no symmetry breaking or dominance detection (Week 5), and no MiniZinc or Gurobi – this week stays CP-SAT only, exactly as Weeks 1 and 2 did. It sets no optimization objective anywhere: every problem this week remains a pure satisfaction task, find one solution or prove none exists, with optimization objectives returning properly in Week 8. Nor does it re-derive AC-3's mechanics or the node/arc/GAC taxonomy – that is Week 2's territory, referenced here by name only, as prior knowledge each unit's opening box assumes.

3.1 Backtracking and the Search Tree

1 • Pre-instruction **Why It Matters**

Placing one queen per row on an 8×8 board already has 8^8 – over 16 million – raw ways to do it before you even start checking for attacks; trying every one, then checking, is hopeless past a small board. Backtracking never builds most of those boards at all: the moment a partial placement is already broken, it stops right there and tries something else. This unit is the algorithm behind that shortcut.

1 • Pre-instruction **Prior Knowledge**

Week 1's N-queens paper model (`Q1`..`Q4`, one variable per row, `AllDifferent` plus the two diagonal conditions) and Week 2's AC-3 and its propagation fixpoint – the idea of narrowing a domain *before* committing to a value – are both assumed here. Backtracking search is the other half of every CSP solver: propagation narrows domains, search decides what to try next when propagation alone cannot finish the job.

Week 1 handed you a formal CSP and a solver that, from your point of view, simply returned an answer. Week 2 opened part of that box: it showed how a solver

narrows every variable's domain before it commits to anything, using AC-3's worklist and `Revise` step. Neither week explained what happens once propagation alone cannot finish the job – once every domain is as narrow as propagation can make it, but more than one value still remains somewhere, and a genuine choice has to be made. This unit is that missing piece: the systematic procedure a solver falls back on once it has to guess.

3.1.1 Chronological backtracking

Backtracking search: Chronological depth-first search over partial assignments: when the current partial assignment cannot be extended with any value of the current variable, undo the most recent choice and try the next alternative at the previous choice point.

Backtracking search is depth-first search over partial assignments to a CSP's variables. At each step, the search holds a *partial assignment* – values already committed to some variables, with the rest still unassigned – and proceeds as follows: choose the next unassigned variable (this unit uses a fixed, arbitrary order, decided in advance; Unit 2 revisits the choice of order itself), try each value in that variable's domain in turn, and for each candidate value check whether the resulting partial assignment is still consistent – satisfies every constraint that touches only variables already assigned. If it is consistent, the search recurses: it moves on to the next unassigned variable with this new value locked in. If it is not consistent, the search simply discards that value and tries the next one in the current variable's domain.

> **Definition: Backtracking search**
>
> Chronological depth-first search over partial assignments: when the current partial assignment cannot be extended with any value of the current variable, undo the most recent choice and try the next alternative at the previous choice point.

> **Analogy**
>
> Backtracking search resembles filling in a Sudoku grid cell by cell in pencil: you write a candidate digit, and the moment it conflicts with a digit already in the same row, column or box, you erase just that one digit and try the next candidate – you never erase the whole grid and start over. Every other cell you had already filled in stays exactly as it was; only the one choice that just failed gets undone.

A word worth pausing on in the definition above is "only variables already assigned." Backtracking never inspects a variable that has not yet been reached along the current branch – it checks the partial assignment built so far against every constraint that partial assignment can already violate, and nothing more. That might look like a weakness – surely a smarter check would catch trouble sooner? – but it is exactly this narrow, look-back-only check that keeps a single consistency test cheap; Unit 3 shows how look-ahead techniques add proactive checking on top of this same loop, without changing the loop itself.

3.1.2 The search tree

search tree: The tree whose root is the empty assignment, whose internal nodes are partial assignments together with a choice of next variable, whose edges are candidate value choices, and whose leaves are either a complete consistent assignment (a solution) or a pruned dead end.

Every run of backtracking search traces out a **search tree**, whether or not anyone ever draws it. The tree's root is the empty assignment – no variable has a value yet. An internal node is a partial assignment together with a choice of which variable to branch on next. Each edge leaving a node is one candidate value tried for that node's variable. A leaf is one of two things: a complete, consistent assignment –

a *solution* – or the endpoint of a branch every one of whose candidate values has already been tried and rejected.

Definition: Search tree

The tree whose root is the empty assignment, whose internal nodes are partial assignments together with a choice of next variable, whose edges are candidate value choices, and whose leaves are either a complete consistent assignment (a solution) or a pruned dead end.

That second kind of leaf has a name of its own: **dead end (failure node)**, a node at which every value in the current variable's domain has been tried and rejected. Reaching a dead end forces the search to *backtrack*: undo the most recent choice and try the previous variable's next untried value.

dead end (failure node): A search-tree node at which every value in the current variable's domain has been tried and rejected, forcing the search to backtrack to the previous choice point.

Definition: Dead end (failure node)

A search-tree node at which every value in the current variable's domain has been tried and rejected, forcing the search to backtrack to the previous choice point.

Analogy

A search tree with a dead end works like following a marked route on a campus map: reaching a hallway that turns out to be a dead end does not mean the whole trip failed – you retrace your last few steps to the previous junction and take a different corridor. The trip only fails for good once every corridor out of every junction, all the way back to the building entrance, has already been tried.

3.1.3 A worked trace: 4-queens

Take Week 1's "one queen per row" 4-queens model: variables `Q1`..`Q4`, each with domain $\{1, 2, 3, 4\}$, under `AllDifferent` on the four columns plus the two diagonal conditions. Fix the variable order `Q1,Q2,Q3,Q4` and the value order ascending, $1, 2, 3, 4$, and trace the search by hand.

With $Q1 = 1$: trying $Q2 = 1$ is rejected outright (same column as $Q1$); $Q2 = 2$ is rejected outright too (shares a diagonal with $Q1 = 1$); $Q2 = 3$ is accepted, but now every value of $Q3$ – $1, 2, 3, 4$ in turn – fails against $Q1, Q2$, so $Q3$'s domain is exhausted: **dead end 1**. Backtracking to $Q2$, its next value $Q2 = 4$ is accepted; then $Q3 = 2$ is accepted; but every value of $Q4$ fails in turn: **dead end 2**. $Q3$'s remaining values, $1, 3, 4$, are then tried and also fail, so $Q2 = 4$'s own branch closes as exhausted. With $Q2$'s domain now fully exhausted – 1 and 2 rejected outright, 3 and 4 each led to a dead end – $Q1 = 1$ itself becomes **dead end 3**, and the search backtracks all the way to the root.

With $Q1 = 2$: $Q2 = 1$ is rejected (diagonal); $Q2 = 2$ is rejected (column); $Q2 = 3$ is rejected (diagonal); $Q2 = 4$ is accepted. $Q3 = 1$ is accepted. $Q4 = 1$ and $Q4 = 2$ are both rejected, but $Q4 = 3$ is accepted and consistent with every earlier variable – a **solution**: $Q1 = 2, Q2 = 4, Q3 = 1, Q4 = 3$.

Summarizing the whole trace: exactly three dead ends, all under the $Q1 = 1$ branch; the $Q1 = 2$ branch succeeds on its very first fully explored path. Figure 3.1 draws this exact trace as a single tree, so you can see the whole shape at a glance instead of only following it step by step in prose.

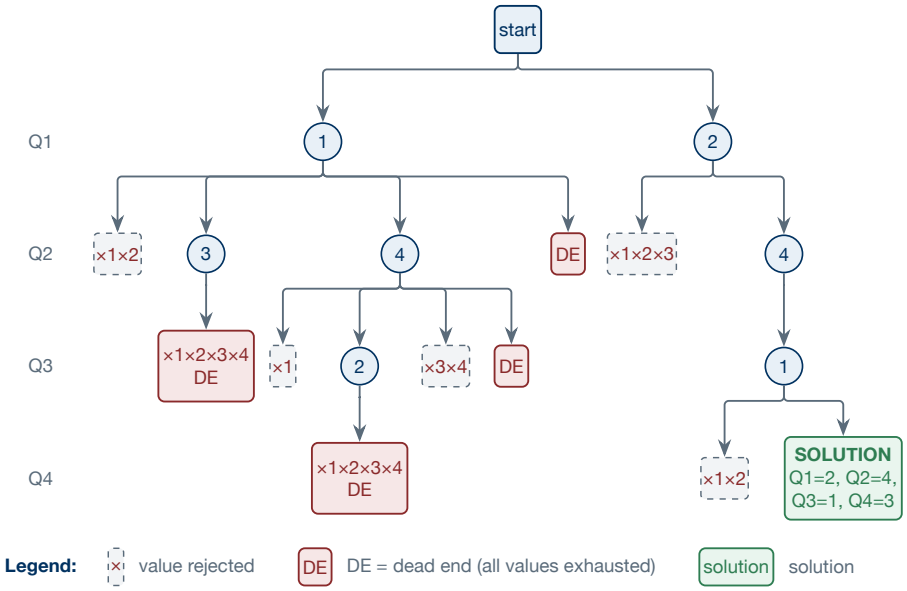


Figure 3.1: The full backtracking search tree for 4-queens under variable order $Q1, Q2, Q3, Q4$ and ascending value order – three dead ends under $Q1 = 1$, the solution $Q1 = 2, Q2 = 4, Q3 = 1, Q4 = 3$ under $Q1 = 2$.

3.1.4 Why backtracking is complete

completeness: A search algorithm is complete if it is guaranteed to find a solution when one exists, or correctly report infeasibility when none does.

Because backtracking systematically tries every value at every level of the tree – skipping only branches already proven inconsistent – it is guaranteed to eventually either find a solution or correctly prove that none exists. This property is worth a name: **completeness**. A CSP is only declared infeasible once the *root itself* becomes a dead end – every value of every variable exhausted along every branch, all the way back to the empty assignment. Nothing about the 4-queens trace above stopped short of that standard; it simply happened to find a solution before the root ever needed to become a dead end.

A common misconception at this point is treating any single dead end as evidence that the whole problem has no solution. It is not: a dead end only closes off *one* branch, and backtracking's guarantee rests on continuing to the *next* untried alternative at the previous choice point, not on stopping the moment one branch fails. The 4-queens trace above hit three dead ends under $Q1 = 1$ alone, and the problem still had a solution waiting under $Q1 = 2$ – infeasibility is a claim about the *entire* tree, never about one branch of it.

One contrast is worth flagging here and no further: this completeness guarantee is exactly what *local search* – Week 4's subject – gives up in exchange for speed on much larger instances; local search can report a good solution quickly, but it cannot, in general, prove that none exists.

3 • Participation **Now You Try**

In the 4-queens hand trace above, after assigning $Q1 = 1, Q2 = 3$, every value tried for $Q3$ fails. Name the operation the search performs next, and explain in one sentence why it does not simply declare the whole problem infeasible at that point.

Feedback. The operation is *backtrack*: the search returns to the previous choice

point, $Q2$, and tries $Q2$'s next untried value ($Q2 = 4$). It does not declare infeasibility because $Q1 = 1, Q2 = 3$ is only one branch out of the whole tree; infeasibility is only correctly declared once every value at every level has been exhausted, i.e. once the *root* itself becomes a dead end – which is exactly what would have happened here if $Q1 = 2$'s branch had also failed.

Check Yourself 3.1

Backtracking search only ever checks constraints between variables that are already assigned – it never inspects a variable that has not yet been reached in the current branch. Explain, using the notion of a dead end, why this is still enough to guarantee the search eventually either finds a solution or correctly reports that none exists.

5 • Follow-through **Transfer — Where Else?**

Every solver you use for the rest of this course – CP-SAT here, Gurobi's branch-and-bound from Week 7 onward – runs a backtracking-shaped search underneath its more sophisticated machinery. Unit 4 shows you the exact parameters CP-SAT exposes over this same loop; Week 4 replaces this systematic loop with local search for problems where it becomes too slow.

3.2 Variable and Value Ordering Heuristics

1 • Pre-instruction **Why It Matters**

In Unit 1's 4-queens trace, all three dead ends happened under $Q1 = 1$ – the search wasted an entire sub-tree before ever reaching the branch that actually works. What if, at every choice point, you could pick the variable and value least likely to lead down a doomed branch?

1 • Pre-instruction **Prior Knowledge**

Unit 1's worked 4-queens trace and its three dead ends – the concrete waste this unit targets – are assumed here, together with a light callback to Week 2's weak-versus-strong-model idea: that unit changed how much a single model tells the solver up front; this unit changes the *order* the solver uses that same information in, a related but distinct lever.

Unit 1 fixed a variable order (`Q1,Q2,Q3,Q4`) and a value order (ascending) in advance, arbitrarily, before the search ever started. Nothing in backtracking's definition requires that; the order was simply the easiest one to describe by hand. This unit asks whether a smarter order can be chosen instead – one that reacts to the current state of the search rather than being fixed in advance – and shows that the answer is emphatically yes, without changing backtracking's algorithm at all.

3.2.1 Variable ordering heuristics: which variable next

variable ordering heuristic: A rule that decides which not-yet-assigned variable the search branches on next.

A **variable ordering heuristic** decides which unassigned variable the search branches on next, replacing Unit 1's fixed order with a rule that consults the current partial assignment.

Definition: Variable ordering heuristic

A rule that decides which not-yet-assigned variable the search branches on next.

Minimum Remaining Values (MRV): A variable-ordering heuristic that always selects the unassigned variable with the fewest values left in its current domain ("fail-first").

The best-known variable ordering heuristic is **Minimum Remaining Values (MRV)**, also called most-constrained-variable: always branch on the unassigned variable with the fewest legal values left in its current domain. MRV embodies what is usually called the *fail-first* principle: if a branch is doomed, better to discover it high in the tree, before wasted work accumulates beneath it. A variable with only one or two remaining values is far more likely to force a dead end soon than a variable with five remaining values, so MRV pushes the search toward discovering that failure early, rather than deep inside a large sub-tree built on top of it.

Definition: Minimum Remaining Values (MRV)

A variable-ordering heuristic that always selects the unassigned variable with the fewest values left in its current domain ("fail-first").

Analogy

MRV resembles scheduling the exam with the fewest open rooms or time slots first: if a clash is coming, you want to discover it while placing the most constrained exam, not after nine easy ones are already locked in and the room you needed for the tenth has quietly disappeared.

degree heuristic: A variable-ordering heuristic (often used to break MRV ties) that selects the unassigned variable involved in the most constraints with other unassigned variables.

MRV can tie – more than one unassigned variable can share the smallest domain size. The **degree heuristic** is the classic tie-breaker: pick the unassigned variable involved in the most constraints with other unassigned variables, a proxy for "how much this choice will restrict the rest of the problem." The degree heuristic is occasionally used on its own, as a primary heuristic, before any domain has been narrowed at all and MRV therefore has nothing to distinguish between variables yet – but its most common role is breaking MRV ties.

Definition: Degree heuristic

A variable-ordering heuristic (often used to break MRV ties) that selects the unassigned variable involved in the most constraints with other unassigned variables.

Analogy

The degree heuristic resembles choosing, between two equally scarce time slots, the course that shares more students with other still-unscheduled courses: it is the placement whose ripple effects reach the most remaining decisions, so settling it first narrows down the most of what is left to plan.

dom/wdeg (domain-over-weighted-degree): An adaptive variable-ordering heuristic that divides a variable's current domain size by a weight accumulated on its constraints each time one of them has caused a search failure, so variables touching historically troublesome constraints are picked earlier.

A third heuristic, **dom/wdeg (domain-over-weighted-degree)**, adapts as the search runs rather than only looking at a fixed snapshot. It divides a variable's current domain size by a *weight* accumulated on each of its constraints, a weight that grows by one every time that particular constraint has caused a search failure so far. Constraints that have caused trouble before get heavier weight, so the variables that touch them get picked earlier the next time a choice is needed. Unlike MRV or the degree heuristic, both of which only ever look at the current snapshot of domain sizes and remaining constraints, dom/wdeg (introduced by Boussemart et al.) is a heuristic that *learns from the search's own history* as it runs.

Definition: dom/wdeg (domain-over-weighted-degree)

An adaptive variable-ordering heuristic that divides a variable's current domain size by a weight accumulated on its constraints each time one of them has caused a search failure, so variables touching historically troublesome constraints are picked earlier.

Analogy

dom/wdeg resembles a budget office that watches most closely whichever expense category has triggered the most over-budget alarms in past quarters: categories that have caused trouble before get weighted more heavily going forward, exactly as dom/wdeg weights a constraint more heavily every time it has caused a search failure.

3.2.2 Value ordering heuristics: which value first

value ordering heuristic: A rule that decides, once a variable has been chosen, in which order to try its domain's values.

Once a variable has been chosen, a separate question remains: in which order should its domain's values be tried? A **value ordering heuristic** answers exactly that. The best-known one is **Least Constraining Value (LCV)**: try first the value that rules out the fewest remaining choices for neighbouring unassigned variables. LCV embodies the *succeed-first* principle – the deliberate opposite intention from MRV. MRV wants to fail fast, so that a doomed branch is discovered as early as possible; LCV wants to succeed fast, by keeping as many options open as possible for the rest of the search once a variable has already been committed to.

Least Constraining Value (LCV): A value-ordering heuristic that tries first the value ruling out the fewest remaining choices for neighbouring unassigned variables ("succeed-first").

Definition: Value ordering heuristic

A rule that decides, once a variable has been chosen, in which order to try its domain's values.

Definition: Least Constraining Value (LCV)

A value-ordering heuristic that tries first the value ruling out the fewest remaining choices for neighbouring unassigned variables ("succeed-first").

Analogy

LCV resembles choosing the travel connection that leaves the most onward options open for the rest of your itinerary, rather than the one that happens to be cheapest right now but locks you into a single remaining route for the rest of

the trip.

Case Study 3.1: LCV on a small neighbourhood

Variable X has domain $\{1, 2, 3\}$ and two unassigned neighbours: Y, domain $\{1, 2, 3\}$, under $X \neq Y$, and Z, domain $\{2, 3\}$, under $X \neq Z$. LCV asks, for each candidate value of X, how many choices it rules out across Y and Z combined. Trying $X = 1$: it removes 1 from Y's domain (one choice ruled out) but Z's domain does not even contain 1, so nothing is ruled out there – **one** choice ruled out in total. Trying $X = 2$: it removes 2 from both Y and Z – **two** choices ruled out. Trying $X = 3$: symmetric to $X = 2$, also **two** choices ruled out. LCV therefore orders X's values as $1, 2, 3$ (or $1, 3, 2$ – the tie between 2 and 3 can break either way), trying $X = 1$ first because it leaves the most remaining options open for Y and Z.

3.2.3 Composing the two: order, not shape

Variable-ordering heuristics answer "which variable next"; value-ordering heuristics answer "which value first." They are not competitors – they compose. A common and effective combination is to choose the variable by MRV, then order that variable's values by LCV, using both heuristics' different goals together at each single choice point. It helps to be precise about what changes and what does not when a heuristic is introduced: the search tree's *shape* stays the problem's shape – the same CSP has the same set of solutions and the same set of dead ends no matter what order they are discovered in. Only the *order* nodes are visited in changes – but that change in order is often a dramatically smaller explored tree in practice (occasionally, on an unlucky instance, a slightly larger one, since no ordering heuristic is guaranteed to help on every possible instance). Figure 3.2 sketches this effect schematically: the same underlying problem, reaching the same solution, explored through a visibly bushier tree under an arbitrary order than under MRV plus LCV.

Tip

None of this changes backtracking's completeness guarantee from Unit 1. An ordering heuristic only changes which value is tried *first* at each choice point – if that value fails, the search still tries every remaining value in turn, exactly as before. A faster route to the same answer is not the same thing as a different answer.

3 • Participation **Now You Try**

During a partial search over a graph-coloring CSP, three unassigned variables remain: A (domain size 3, touches 2 other still-unassigned variables), B (domain size 1, touches 3 others), C (domain size 2, touches 1 other). Which variable does MRV select next?

Feedback. MRV picks the variable with the smallest current domain – B, with domain size 1 – regardless of its degree. No tie exists here, so the degree heuristic is not needed as a tie-break in this particular case. If two variables had tied on domain size instead, the degree heuristic would break the tie toward whichever

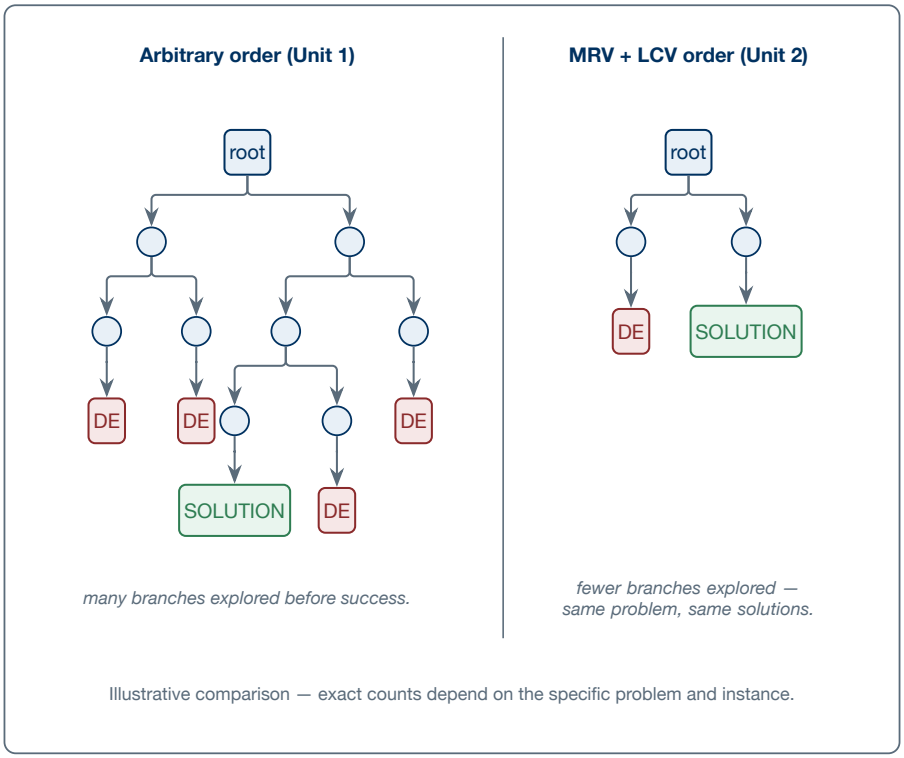


Figure 3.2: Schematic comparison of the same CSP explored under an arbitrary order (left) and under MRV plus LCV (right) – same problem, same solutions, a visibly smaller explored tree on the right.

one touches more remaining unassigned variables.

Check Yourself 3.2

Three unassigned variables remain in a partial search: X has domain size 2 and is directly constrained with 3 other unassigned variables; Y has domain size 2 and is directly constrained with 1 other; Z has domain size 4 and is directly constrained with 2 others. Which variable does MRV select? If MRV ties, which variable does the degree heuristic pick, and which variable does the search actually branch on next?

5 · Follow-through Transfer — Where Else?

The heuristics you just compared are not folklore – this week's own graph-coloring, magic-square and Langford code (Unit 4) makes the difference between MRV plus LCV and an arbitrary order visible in real running time and real branch counts, not just in a hand trace. dom/wdeg's "learn from past failures" idea is also a direct preview of Week 4's nogood learning, which generalizes the same principle into a full learning mechanism.

3.3 Forward Checking and MAC

1 • Pre-instruction — Why It Matters

Forward checking and MAC do not need a new algorithm – they need something already built in Week 2: AC-3's own propagation routine, invoked from inside the search instead of only once before it starts.

1 • Pre-instruction — Prior Knowledge

Week 2's local consistency (node/arc/generalized arc consistency) and AC-3 and its propagation fixpoint are assumed here; this unit reuses AC-3 exactly as built there. The only change is *when* it runs: once before search began in Week 2, and now after every single assignment during search.

Plain backtracking, as Unit 1 defined it, is "look-back only": it checks constraints against already-assigned variables, and it only discovers a failure once every value of the *current* variable has been tried – it never proactively narrows any other variable's domain. Unit 2 changed the *order* of the search tree, but left the tree's size, in principle, untouched. This unit changes *how much of the tree exists in the first place*, by pruning other variables' domains before a doomed branch is even tried – a genuinely different lever from either of the first two units'.

3.3.1 Forward checking: a one-hop look ahead

Forward checking: A look-ahead technique that, immediately after a variable is assigned, removes now-inconsistent values from the domains of its directly constrained unassigned neighbours, backtracking at once if any neighbour's domain becomes empty.

Forward checking works as follows: the instant a variable is assigned, remove every now-inconsistent value from the domain of each *directly* constrained still-unassigned neighbour. If any of those neighbouring domains becomes empty, backtrack immediately – without waiting to exhaust the emptied-domain variable's values one by one, the way plain backtracking (Unit 1) would have.

Definition: Forward checking

A look-ahead technique that, immediately after a variable is assigned, removes now-inconsistent values from the domains of its directly constrained unassigned neighbours, backtracking at once if any neighbour's domain becomes empty.

Analogy

Forward checking resembles crossing a booked room off every other still-unscheduled exam's shortlist the moment you book it for one exam: everyone else's remaining options are updated immediately, instead of discovering the clash only when you later try to book that same room again for a different exam.

The phrase "directly constrained" is doing real work in that definition. Forward checking is a *one-hop* check outward from the just-assigned variable – it removes values from that variable's own neighbours, and stops there. It does not propagate any further through the rest of the constraint graph, even if a neighbour's domain shrinks enough to affect a value two hops away.

3.3.2 MAC: maintaining arc consistency during search

Maintaining Arc Consistency (MAC): A look-ahead technique that, after every assignment, re-establishes full arc consistency (via AC-3) over all remaining unassigned variables, propagating beyond just the newly assigned variable's direct neighbours.

Maintaining Arc Consistency (MAC) goes further: after every assignment, re-run full arc consistency – Week 2's AC-3, unchanged – over all remaining unassigned variables, not just the one-hop neighbours forward checking stops at. Because MAC's propagation step is exactly AC-3, it inherits AC-3's own guarantee: it keeps revising arcs, re-queueing affected neighbours, until no further removal anywhere is possible – reaching the same kind of fixpoint Week 2 defined, but now triggered after every single search-tree assignment instead of only once before search began.

Definition: Maintaining Arc Consistency (MAC)

A look-ahead technique that, after every assignment, re-establishes full arc consistency (via AC-3) over all remaining unassigned variables, propagating beyond just the newly assigned variable's direct neighbours.

Analogy

MAC resembles a lab's full department-wide rebooking system that, after every new booking, re-checks and updates every other still-open booking's shortlist, not just the one directly affected – more thorough than forward checking's one-hop update, at the cost of rechecking more of the schedule each time a single booking changes.

Forward checking and MAC are the classic "cheap and shallow" versus "expensive and thorough" look-ahead trade-off: MAC strictly dominates forward checking in how much it prunes – anything forward checking removes, MAC removes too, plus whatever its further propagation catches – but MAC also does strictly more work at every single search-tree node.

Case Study 3.2: Forward checking versus MAC on a path CSP

Three variables A, B, C sit on a path: $A \neq B$, $B \neq C$, and critically, *no direct constraint between A and C*. All three domains start at $\{1, 2\}$. Assign $A = 1$.
Forward checking looks only at A's direct neighbour, B: it removes 1 from B's domain (since $A \neq B$), leaving $B = \{2\}$. C is not a direct neighbour of A, so forward checking leaves C completely untouched at this step: $C = \{1, 2\}$.
MAC starts the same way – $B = \{2\}$ after the first `Revise` – but does not stop there. Because B's domain just changed, MAC's AC-3 worklist re-queues the arc (C, B): with B now forced to the single value 2, C's only remaining supporting value against $B \neq C$ is 1, so MAC removes 2 from C's domain, leaving $C = \{1\}$ – a narrowing forward checking misses entirely at this step, precisely because C is two hops from A, not one.

Figure 3.3 shows this same worked example as a before/after domain table, so the gap between the two techniques is visible at a glance.

A common misconception at this point is assuming forward checking is simply "a smaller version of MAC" that always catches most of what MAC catches. The worked example above shows this is not reliably true: forward checking left C's domain completely unchanged, at exactly the same search-tree node where MAC already knew C could only be 1. Whether that gap matters depends entirely on the

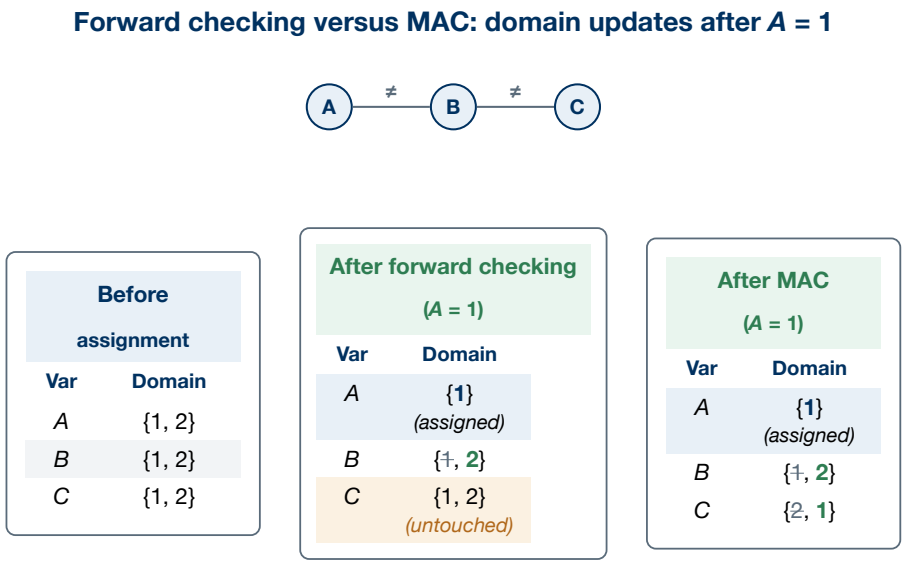


Figure 3.3: Domains for A, B, C ($A \neq B$, $B \neq C$, no direct A–C constraint) before assignment, after forward checking on $A = 1$, and after MAC on $A = 1$.

constraint graph's shape – a variable two or more hops away from every currently-assigned variable is invisible to forward checking no matter how tightly constrained it secretly already is.

Complexity (big-O)

Per node, forward checking costs at most $O(d)$ work for each directly constrained unassigned neighbour of the just-assigned variable – checking each of that neighbour's up-to-d remaining values once against the single new assignment. MAC, by re-running AC-3 itself at every node, costs up to AC-3's own worst-case bound, $O(e \cdot d^3)$ (Week 2), where e is the number of remaining constraints and d the largest remaining domain size. MAC's extra propagation work can still pay off overall despite this higher per-node cost, because it can prune whole subtrees – like the doomed values of C above – one level earlier than forward checking would have discovered them.

3 • Participation **Now You Try**

Using the A, B, C example above ($A \neq B$, $B \neq C$, domains $\{1, 2\}$ each), compute B's and C's domains immediately after forward checking runs on $A = 1$.

Feedback. Forward checking removes 1 from B (since $B \neq A$), giving $B = \{2\}$. C is not directly constrained with A, so forward checking leaves it unchanged at this step: $C = \{1, 2\}$. This is exactly the gap MAC closes: MAC would additionally remove 2 from C, leaving $C = \{1\}$, because B's forced value of 2 propagates one hop further than forward checking reaches.

Check Yourself 3.3

In the worked A, B, C example, forward checking failed to narrow C's domain after $A = 1$ was assigned, while MAC did. Explain, in general terms, why MAC's extra propagation work at every node can still pay off overall, even though each individual node costs more to process than under forward checking.

5 • Follow-through **Transfer — Where Else?**

Unit 4's CP-SAT decision-strategy parameters sit on top of a solver whose internal engine already performs something closer to MAC than to plain forward checking, on every one of its global constraints, at every single node – you are about to see how little manual look-ahead code you personally have to write, precisely because CP-SAT is already doing Units 1–3's work internally.

3.4 Search Parameters in CP-SAT

1 • Pre-instruction **Why It Matters**

You have hand-traced backtracking, reasoned about which variable and value to try, and reasoned about forward checking versus MAC – CP-SAT already runs all of that, automatically, every time you call `solver.Solve(model)`. This unit is the handful of parameters that let you see, measure, and occasionally steer that hidden machinery on a problem hard enough to actually need it.

1 • Pre-instruction **Prior Knowledge**

This week's own Units 1–3 – backtracking and search-tree vocabulary, MRV/degree/dom-wdeg/LCV naming, forward checking/MAC naming – supply the vocabulary this unit's parameter names map onto. Week 1 Unit 4's CP-SAT basics (`CpModel`, `NewIntVar`, `Solve`, `AddAllDifferent`) supply the API this unit builds on.

Every model you have built so far in this course was solved by calling `solver.Solve(model)` and simply reading off the result – exactly the black box Week 1 promised to leave closed until this point. This unit finally opens it on the CP-SAT side specifically: three concrete parameters that let you configure, observe and compare the search underneath any model you have already built, using two new, larger problems – a 10-node graph-coloring instance and a fresh combinatorial puzzle, Langford pairs – chosen specifically because their search effort is large enough for variable/value order to leave a visible mark, unlike the tiny toy instances used earlier in this course.

3.4.1 Decision strategies: telling the solver which rule to use

decision strategy (CP-SAT): A model annotation, added with `AddDecisionStrategy`, that fixes which variable-selection and value-selection rule CP-SAT's search uses over a given list of variables.

A **decision strategy (CP-SAT)** is a model annotation that fixes which variable-selection and value-selection rule CP-SAT's search uses over a given list of variables. It is added by calling `AddDecisionStrategy` on the model, with three arguments: the list of variables to control, a variable-selection rule, and a value-selection

rule. Three enum values matter for this unit: `CHOOSE_FIRST` (a fixed order over the given variable list – CP-SAT's analogue of Unit 1's arbitrary baseline order), `CHOOSE_MIN_DOMAIN_SIZE` (CP-SAT's built-in analogue of Unit 2's MRV), and `SELECT_MIN_VALUE` or `SELECT_MAX_VALUE` (concrete value choices, standing in for Unit 2's LCV intuition – always try the smallest, or always the largest, remaining value first).

Definition: Decision strategy (CP-SAT)

A model annotation, added with `AddDecisionStrategy`, that fixes which variable-selection and value-selection rule CP-SAT's search uses over a given list of variables.

Analogy

A decision strategy resembles giving a scheduling assistant a stated rule of thumb – "always fill in the tightest-constrained slot first, and when in doubt try the earliest time" – instead of leaving the assistant to use its own default judgement about what to try next.

Note plainly what CP-SAT does *not* expose: there is no literal "degree" or "dom/wdeg" enum value. CP-SAT's own *default* automatic search – reached by never calling `AddDecisionStrategy` at all – already blends several internal heuristics, and usually outperforms a hand-picked strategy. Forcing a strategy is a tool for experimentation, diagnosis and specific structured cases, not a default habit; the comparisons below force a strategy deliberately, precisely to make that internal blending visible by contrast.

Caution

`AddDecisionStrategy` only takes effect once the solver's own `search_branching` parameter is also set to `FIXED_SEARCH`. Without that second line, CP-SAT's automatic search silently ignores the forced strategy and falls back to its own internal blend – no error, no warning, just a strategy that quietly never ran. Every ordered configuration below sets both lines together.

This week's first coded problem is the Petersen graph's 3-colouring: 10 nodes (an outer 5-cycle, an inner pentagram, and five spokes joining them, 15 edges total, chromatic number 3), one integer variable per node, and one pairwise `!=` constraint per edge – structurally harder than Week 1's 7-node Australia map-coloring toy, deliberately, so that ordering choices leave a visible mark:

```
colours = [model.NewIntVar(0, 2, f"colour_{node}") for node in range(
    NUM_NODES)]
return colours
```

```
for node_a, node_b in EDGES:
   model.Add(colours[node_a] != colours[node_b])
```

The default configuration below runs CP-SAT with no forced strategy at all, using a fixed random seed so the comparison is repeatable, and reads its search statistics through a small shared helper,

```
print(f"Status: {solver.StatusName(status)}")
print(f"NumBranches: {solver.NumBranches()}")
```

```
print(f"NumConflicts: {solver.NumConflicts()}")
print(f"WallTime: {solver.WallTime():.4f}")
```

imported by every problem file this week:

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0009
```

The forced configuration adds `AddDecisionStrategy` over the same ten colour variables, forcing `CHOOSE_MIN_DOMAIN_SIZE` for variable choice and `SELECT_MIN_VALUE` for value choice, together with the required `FIXED_SEARCH` line from the warning above:

```
model.AddDecisionStrategy(
    colours, cp_model.CHOOSE_MIN_DOMAIN_SIZE, cp_model.SELECT_MIN_VALUE
)
solver = cp_model.CpSolver()
solver.parameters.search_branching = cp_model.FIXED_SEARCH
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 65
NumConflicts: 0
WallTime: 0.0008
```

Both real captured runs solve the Petersen graph, and on this instance the forced MRV-plus-min-value strategy explored a slightly smaller tree than the default – 65 branches and no conflicts, against 69 branches and 1 conflict for the default. The gap is modest here, exactly the kind of small, real difference an ordering choice produces on a moderately hard instance, not a dramatic one – Unit 4's Langford comparison below shows a case where forcing an order does the opposite.

3.4.2 Search workers: a light preview of CP-SAT's parallel portfolio

search worker (CP-SAT): One of several parallel threads CP-SAT can run at once on the same model, set via a worker-count parameter.

The solver parameter **search worker (CP-SAT)** `num_search_workers` configures a worker count – how many parallel threads CP-SAT runs at once on the same model, each internally free to explore the search differently. This unit stops at observing the wall-clock effect of changing this one number; *why* different workers help at all, and how they share information as they run, is Week 4's job.

Definition: Search worker (CP-SAT)

One of several parallel threads CP-SAT can run at once on the same model, configured with the `num_search_workers` parameter.

Analogy

Search workers resemble handing the same timetable-building task to several assistants at once, each trying a different rule of thumb, and taking whichever one finishes first – without yet asking how the assistants might have shared notes along the way.

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = num_workers
solver.parameters.random_seed = 42
status = solver.Solve(model)
print_search_stats(solver, status)
```

Expected Output

```
num_search_workers=1
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0008

num_search_workers=8
Status: OPTIMAL
NumBranches: 69
NumConflicts: 1
WallTime: 0.0024
```

Read this real captured pair honestly: on this particular instance, 8 workers did not run faster than 1 – wall time actually went up, since starting and coordinating eight threads carries its own overhead, and the Petersen graph solves in under a millisecond regardless. That is not a mistake in the code; it is exactly what you should expect on an instance this small, where parallel search overhead outweighs any speedup it might otherwise provide. The benefit of extra workers shows up on genuinely hard instances, not on ones a single thread already solves near-instantly – a distinction Week 4 develops further.

3.4.3 Time limits and reading a cut-short search

time limit (solver parameter): A wall-clock cutoff, set via a time-limit parameter, after which the solver returns its best finding so far instead of continuing to search.

The solver parameter `max_time_in_seconds` sets a **time limit (solver parameter)** – a hard wall-clock cutoff, after which the solver returns whatever it has found so far instead of continuing to search. `solver.StatusName(status)` reports what that return actually means: `OPTIMAL` or `FEASIBLE` if a solution was confirmed before time ran out, or `UNKNOWN` if the cutoff hit before the search could prove anything at all, feasible or not.

Definition: Time limit (solver parameter)

A wall-clock cutoff, set via the `max_time_in_seconds` parameter, after which the solver returns its best finding so far instead of continuing to search.

Analogy

A time limit resembles telling the scheduling assistant "report the best timetable found within the next ten minutes," even if you have not finished checking every possibility – you get whatever progress was made, honestly reported, rather than an answer you have to wait indefinitely for.

Langford pairs (Langford sequence): The CSP of arranging the numbers `1..n`, each appearing exactly twice, in a sequence of length `2n` such that there are exactly `k` other entries between the two occurrences of every value `k`.

This is where **Langford pairs (Langford sequence)**, this week's new problem, enters. Arrange the numbers $1..n$, each appearing exactly twice, into a sequence of length $2n$, such that there are exactly k other entries between the two occurrences of every value k (equivalently, their two positions differ by $k+1$). A solution exists only when $n \equiv 0$ or $n \equiv 3 \pmod 4$; this unit uses $n = 7$ ($7 \equiv 3 \bmod 4$, so a solution exists). Langford pairs was chosen specifically because its search-tree size is dramatically sensitive to variable and value order – exactly the property this unit needs real numbers to demonstrate.

Definition: Langford pairs (Langford sequence)

The CSP of arranging the numbers $1..n$, each appearing exactly twice, in a sequence of length $2n$ such that there are exactly k other entries between the two occurrences of every value k.

```
positions = [
    [model.NewIntVar(0, NUM_POSITIONS - 1, f"pos_{k}_{i}") for i in range(2)]
    for k in range(1, N + 1)
]
return positions
```

Each value k gets two position variables, ranging over all $2n$ sequence slots; `AllDifferent` over every one of the $2n$ position variables forces every value into a distinct slot, and one further constraint per value k pins the exact spacing the definition demands:

```
model.AddAllDifferent([entry for pair in positions for entry in pair])
for k in range(1, N + 1):
    first, second = positions[k - 1]
    model.Add(second - first == k + 1)
```

Both the default and the forced-strategy configurations below set a generous `max_time_in_seconds=10` safety cap – never actually triggered, since both finish in about two milliseconds – to guarantee the comparison always terminates even on a machine slower than the one that captured these real numbers:

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
solver.parameters.max_time_in_seconds = 10
status = solver.Solve(model)
print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 564
NumConflicts: 21
WallTime: 0.0020
```

```
model.AddDecisionStrategy(
   flat, cp_model.CHOOSE_MIN_DOMAIN_SIZE, cp_model.SELECT_MIN_VALUE
)
solver = cp_model.CpSolver()
solver.parameters.search_branching = cp_model.FIXED_SEARCH
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
solver.parameters.max_time_in_seconds = 10
status = solver.Solve(model)
print_search_stats(solver, status)
```

Expected Output

```
Status: OPTIMAL
NumBranches: 843
NumConflicts: 0
WallTime: 0.0025
```

Read this pair honestly, too: on this particular Langford(7) instance, the forced `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MIN_VALUE` strategy explored *more* branches than CP-SAT's own default automatic search – 843 against 564 – even though it reached zero conflicts along the way. This is not a contradiction of anything Unit 2 taught; it is the exact case this unit's opening paragraph warned about: CP-SAT's default automatic search already blends several internal heuristics, tuned against a huge range of real instances, and it can beat a single hand-picked strategy on a given problem. Forcing a strategy remains useful for the reasons already given – experimentation, diagnosis, reproducibility – but "forced" does not mean "guaranteed better."

To see what a genuinely tight cutoff looks like, rather than the generous 10-second safety cap above, a separate run caps the same model at a mere 0.001 seconds:

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
solver.parameters.max_time_in_seconds = time_limit_seconds
status = solver.Solve(model)
print_search_stats(solver, status)
```

Expected Output

```
Status: UNKNOWN
NumBranches: 0
NumConflicts: 0
WallTime: 0.0010
```

At this problem's tiny scale, model transfer and presolve alone can consume the entire time budget before the search ever takes its first branch – the captured `NumBranches: 0` above is a real, honest result of that, not an error in the code.

`StatusName` reports `UNKNOWN` rather than `INFEASIBLE` precisely because the solver never got far enough to prove anything either way; it is reporting "ran out of time," not "no solution exists." Figure 3.4 plots all three real captured branch counts side by side.

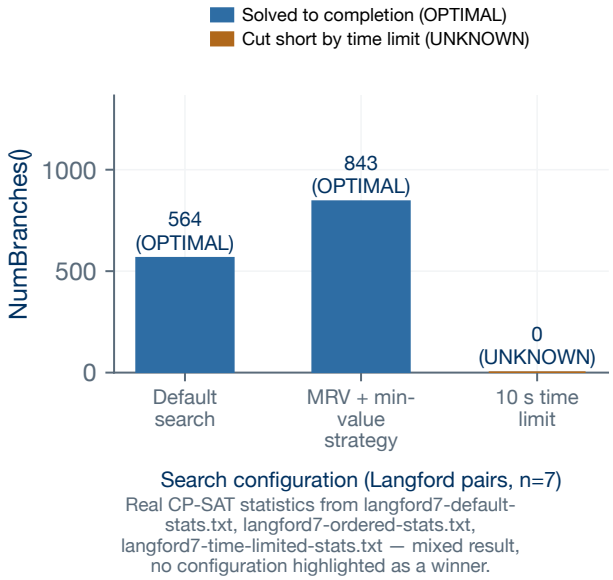


Figure 3.4: Real captured `NumBranches` for Langford(7) under CP-SAT's default search, a forced MRV-plus-min-value decision strategy, and a 0.001-second time limit.

3.4.4 Putting it together: the same recipe on a third problem

The exact same recipe – default search against a forced `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MIN_VALUE` strategy, both read through `print_search_stats` – applies unchanged to this week's third coded problem, the 4×4 magic square (16 cells, domain 1–16, `AllDifferent` over every cell plus 10 equal-sum constraints for its four rows, four columns and two diagonals, each equal to the magic constant 34). Magic square was formulated on paper only in Week 1 Unit 3; this is its first CP-SAT code, in `kod_ornekleri/hafta03/modeller/magic_square.py`, following the identical variables/constraints/solve pattern already worked through twice above.

Default search's real captured statistics, `kod_ornekleri/hafta03/ciktilar/magicsquare4-default-stats.txt`:

Expected Output

```
Status: OPTIMAL
NumBranches: 3107
NumConflicts: 53
WallTime: 0.0119
```

The forced strategy's real captured statistics, `kod_ornekleri/hafta03/ciktilar/magicsquare4-ordered-stats.txt`:

Expected Output

```
Status: OPTIMAL
NumBranches: 3067
NumConflicts: 33
WallTime: 0.0114
```

Real captured numbers again, and again a modest rather than dramatic gap: the forced strategy explored slightly fewer branches (3067 against 3107) and noticeably fewer conflicts (33 against 53). Three problems, three honestly different outcomes – a small win for the forced strategy on Petersen, a clear loss on Langford, a small win again on the magic square – is itself this unit's point: a forced decision strategy is a tool to reach for deliberately on a given instance, never a rule you apply blindly and expect to win every time.

3 • Participation **Now You Try**

Run the Langford-pairs model for $n = 7$ once with CP-SAT's default search and once with `AddDecisionStrategy` forcing `CHOOSE_MIN_DOMAIN_SIZE` plus `SELECT_MIN_VALUE` on the position variables (both runs capped at `max_time_in_seconds=10`). Which run's `NumBranches()` is smaller?

Feedback. Comparing the two real captured files, `kod_ornekleri/hafta03/ciktilar/langford7-default-stats.txt` and `kod_ornekleri/hafta03/ciktilar/langford7-ordered-stats.txt`, the *default* run's `NumBranches()` is the smaller one: 564 branches against the forced strategy's 843. On this particular instance, CP-SAT's own automatic search outperformed the hand-forced MRV-plus-min-value strategy – a genuine, captured counter-example to assuming a forced strategy always wins, and exactly why this unit insists a forced strategy is a tool for experimentation and diagnosis, not a default habit.

Check Yourself 3.4

CP-SAT's default automatic search already blends multiple internal heuristics without a single call to `AddDecisionStrategy`. Given this, state one legitimate reason a course exercise – or a real project – might still force a specific decision strategy by hand.

5 • Follow-through **Transfer — Where Else?**

Every parameter this unit exposed – decision strategy, worker count, time limit – is exactly what Week 4 tunes further (restarts, nogood learning, portfolios) and what Week 15 revisits when a model refuses to solve at all (solver tuning, warm starts, hints). This is not a one-week trick; it is the first three knobs of a toolbox used for the rest of the course.

Chapter Summary

Summary

This chapter opened the box the previous two weeks left closed. Unit 1 defined backtracking search as chronological depth-first search over partial assignments, defined the search tree it traces out (root, internal nodes, edges, dead ends, solution leaves), and hand-traced it in full on 4-queens – three dead ends under $Q1 = 1$, a solution under $Q1 = 2$ – to argue why systematic backtracking is complete: a CSP is only declared infeasible once the root itself becomes a dead end. Unit 2 replaced Unit 1's fixed, arbitrary order with heuristics that react to the search's current state: MRV ("fail-first," branch on the fewest remaining values), the degree heuristic (a common MRV tie-break, most constraints with other unassigned variables), dom/wdeg (an adaptive heuristic that learns from past search failures), and LCV ("succeed-first," try the value that rules out the fewest choices for neighbours) – all changing the search tree's *order*, never its underlying shape or set of solutions. Unit 3 moved from reordering the tree to pruning it before a doomed branch is even tried: forward checking's one-hop update of a just-assigned variable's direct neighbours, against MAC's fuller re-propagation via Week 2's own AC-3, worked through on the same A, B, C path example where forward checking left C untouched and MAC caught its narrower domain one step earlier. Unit 4 closed the chapter on CP-SAT's own API: `AddDecisionStrategy` (with the easy-to-miss requirement that `search_branching` must also be set to `FIXED_SEARCH` for it to take effect at all), `num_search_workers`, and `max_time_in_seconds`, read through real captured statistics on three problems – Petersen graph colouring, the 4×4 magic square, and Langford(7) – with an honest, mixed result: a forced ordering strategy won narrowly on Petersen and on the magic square, but *lost* to CP-SAT's own default automatic search on Langford(7), a genuine reminder that forcing a strategy is a tool for experimentation and diagnosis, never a guaranteed win. Next week keeps every model and every parameter this chapter introduced, but trades backtracking's completeness guarantee for speed: local search gives up the guarantee of finding a solution in exchange for handling much larger instances, and restarts with nogood learning generalize dom/wdeg's own "learn from past failures" idea into a full learning mechanism running on top of the systematic search this chapter just opened.

Check Yourself Answers

Question (Unit 1 – backtracking, dead ends and completeness). Backtracking search's guarantee does not come from inspecting variables early – it comes from the systematic order in which it visits the whole tree. Because the search tries every value at every level of the tree, skipping only branches already proven inconsistent, every possible complete assignment is implicitly considered exactly once, in some order. A branch that cannot be extended with any value of the current variable becomes a dead end and triggers backtracking to the previous choice point, where the next untried alternative is tried instead. The entire CSP is only declared infeasible once every value at every level – including the very first variable's – has been exhausted along every branch, i.e. once the root itself becomes a dead end. No complete assignment is ever skipped without being checked, which is exactly why

a dead end several levels deep, like the three under $Q1 = 1$ in the 4-queens trace, is never mistaken for the whole problem's infeasibility.

Question (Unit 2 – MRV, ties and the degree heuristic). MRV first narrows the choice to the smallest current domain size: X and Y both have domain size 2, smaller than Z's domain size 4, so Z is eliminated immediately and MRV ties between X and Y. The degree heuristic breaks that tie in favour of the variable touching more remaining unassigned constraints – X, with 3, over Y, with 1. The search therefore branches on X next: MRV narrows the field, and the degree heuristic, used here exactly as its usual role suggests, resolves what MRV alone left undecided.

Question (Unit 3 – why MAC's extra work can still pay off). MAC's per-node cost is amortized because it can prune whole subtrees earlier than forward checking would. In the worked A, B, C example, MAC catches C's narrowed domain one level higher in the tree than forward checking does – forward checking would only have discovered C's trouble later, once the search actually tried a doomed value for C and failed. On problems where a lot of extra propagation is available this way, MAC visits far fewer nodes overall despite its higher per-node cost. On problems where little extra propagation is available – where most variables really are only one hop from whatever was just assigned – MAC's extra overhead goes largely unrewarded, and forward checking is the cheaper choice. This is exactly why real solvers, as Unit 4's honest, mixed Petersen/magic-square/Langford results show, make the strength of look-ahead a tunable choice rather than a fixed rule applied blindly to every problem.

Question (Unit 4 – why force a decision strategy at all). Forcing a specific strategy is useful for teaching and diagnosis: it lets you observe, in isolation, the effect one of Unit 2's named heuristics has on search-tree size, since the default automatic search mixes several heuristics internally without exposing which one mattered for a given result. It is also useful for reproducibility – a fixed strategy behaves deterministically from run to run, unlike the default portfolio search combined with multiple workers, whose exact path can vary. And occasionally, for a genuinely structured problem where domain knowledge about a good order is already available, forcing that order can avoid the overhead of the solver rediscovering it on its own. None of these reasons claim a forced strategy always wins – Unit 4's own Langford(7) numbers are a real, captured counter-example to that assumption.